DOCKER CHEAT SHEET

2026 EDITION

Build. Ship. Run. Scale.

The Practical Docker Reference for Modern Developers & DevOps Engineers

AlgoTutor

01 HOW TO USE THIS GUIDE

This cheat sheet is designed as a dense, highly practical reference for developers, DevOps engineers, and cloud professionals navigating modern containerized workflows in 2026. It progresses logically from basic principles to advanced production techniques.

Target Audience

- **Beginners:** Focus on Fundamentals, Commands, and Lifecycle.
- **Backend & Full-Stack:** Focus on Dockerfile, Compose, and Networking.
- **DevOps & Cloud Engineers:** Focus on Multi-stage Builds, Security, and Production.
- **Interview Candidates:** Review the Interview Booster and Troubleshooting matrix.

Docker Learning Path

Fundamentals
↓
Images & Containers
↓
Dockerfiles & Builds
↓
Volumes & Networking
↓
Docker Compose
↓
Production & Security

02 DOCKER FUNDAMENTALS

Concept	Definition
Image	A read-only template with instructions for creating a Docker container (Code + Runtime + Libs).
Container	A runnable instance of an image. Isolated, lightweight, and ephemeral by default.
Dockerfile	A text file containing the sequential instructions used to build a Docker image.
Engine / Daemon	The background service (`dockerd`) that manages Docker objects.
Registry	A storage and distribution system for named Docker images (e.g., Docker Hub, AWS ECR).

Core Architecture

[Developer (CLI)]
↓
[Docker Daemon]
↓
(Pulls Images from Registry)
↓
[Runs Containers]
↓
[Attaches Networks & Volumes]

Feature	Containers	Virtual Machines (VMs)
Architecture	Shares the host OS kernel.	Runs a complete guest OS per VM.
Startup Time	Milliseconds to seconds.	Minutes.
Resource Usage	Lightweight (MBs).	Heavy (GBs).
Isolation	Process-level isolation (cgroups/ namespaces).	Hardware-level isolation (Hypervisor).

> **ENGINEERING INSIGHT:** Containers are ephemeral. When a container is removed, any data written to its writable layer is lost. Always use **Volumes** or **Bind Mounts** for persistent data.

03 SYSTEM & CLI BASICS

Command	Purpose	Example / Notes
`docker version`	Show client and server version.	Ensure both Client and Engine are running.
`docker info`	Display system-wide information.	Check storage driver, node limits, active containers.
`docker login`	Log in to a Docker registry.	`docker login ghcr.io`
`docker logout`	Log out from a registry.	Removes credentials from local storage.
`docker search <term>`	Search Docker Hub for images.	`docker search ubuntu --filter stars=100`
`docker system df`	Show docker disk usage.	Displays space used by images, containers, and volumes.

04 IMAGE OPERATIONS

Pull & List

`docker pull <image>:<tag>`

Download an image from a registry.

`docker images` OR `docker image ls`

List locally stored images.

Remove Images

`docker rmi <image_id>`

Remove a specific image. Cannot remove if a container is using it.

`docker image prune -a`

Remove all unused images (not referenced by any container).

Build, Tag & Push

`docker build -t app:v1 .`

Build an image from a Dockerfile in the current directory.

`docker tag app:v1 myrepo/app:v1`

Create a new tag (alias) pointing to the same image ID.

`docker push myrepo/app:v1`

Push the image to a remote registry.

Inspect & History

`docker image inspect <image>`
`docker history <image>`

QUICK TIP — AVOID `latest`: Blindly relying on the `latest` tag in production is risky. It is just a mutable string pointer, not a guarantee of the newest version. Always pin images to specific versions or SHA digests (e.g., `node:18.17.0-alpine`).

05 CONTAINER LIFECYCLE

Command	Description
`docker run [OPTIONS] IMAGE`	Combines `create` and `start`. Runs a process in a new container.
`docker ps` / `docker ps -a`	List running containers. Add `-a` to see stopped containers as well.
`docker stop <container>`	Gracefully stop a running container (sends SIGTERM, then SIGKILL after grace period).
`docker kill <container>`	Force stop a container immediately (sends SIGKILL).
`docker rm <container>`	Remove a stopped container. Use `-f` to force remove a running one.
`docker restart <container>`	Stops and then immediately starts a container.

Common `docker run` Flags

- `-d` : Detached mode (run in background).
- `-it` : Interactive mode + allocate a pseudo-TTY (useful for shells).
- `--name <name>` : Assign a custom name to the container.
- `-p <host_port>:<container_port>` : Map a port from host to container.
- `-v <host_dir>:<container_dir>` : Mount a volume or bind mount.
- `--rm` : Automatically remove the container when it exits.
- `--restart always` : Restart policy (always, on-failure, unless-stopped).

```
# Example: Run Nginx in background, named 'web', mapping port 8080 to 80
docker run -d --name web -p 8080:80 --restart unless-stopped nginx:alpine
```

06 EXEC, LOGS & DEBUGGING

Execution & Inspection

`docker exec -it <container> sh`

Open a shell inside a running container.

`docker inspect <container>`

Return low-level info (JSON) on IPs, mounts, state.

`docker top <container>`

View running processes inside a container.

Logs & Monitoring

`docker logs -f <container>`

Fetch and follow real-time stdout/stderr logs.

`docker stats`

Live stream of CPU, Memory, and Network usage.

`docker events`

Get real-time events from the server (starts, stops).

CONTAINER NOT WORKING? Troubleshooting Flow

Container exits / crashes
↓
Check logs (`docker logs <id>`)
↓
Inspect state (`docker inspect <id>`)
↓
Check env variables & ports
↓
Check filesystem permissions (Volumes)
↓
Verify resource limits (OOMKilled?)

07 NETWORKING

Driver	Description
bridge	(Default). Creates a private internal network. Containers can communicate via IP. **User-defined bridges** allow DNS resolution by container name.
host	Removes network isolation. Container uses the host's networking directly (no port mapping needed). Linux only.
none	Disables all networking for the container.
overlay	Connects multiple Docker daemons together (Swarm/multi-host networking).

Network Commands & Discovery

- `docker network ls` : List all networks.
- `docker network create <name>` : Create a user-defined bridge network.
- `docker network inspect <name>` : See which containers are attached and their IPs.
- `docker network connect <network> <container>` : Attach a running container to a network.

WHY USE CUSTOM NETWORKS? Containers on the default bridge network can only communicate via IP. Containers on a **user-defined network** can communicate using their container names (DNS service discovery).

```
docker network create app-net
docker run -d --name backend --network app-net my-backend
docker run -d --name frontend --network app-net my-frontend
# frontend can now simply HTTP GET http://backend
```

08 VOLUMES & PERSISTENT DATA

Storage Type	Best For	Persistence	Typical Use Case
Volumes	Production data	Managed by Docker in host filesystem (`/var/lib/docker/volumes/`).	Database storage, shared data between containers.
Bind Mounts	Development	Linked to a specific absolute path on the host machine.	Live-reloading source code during local development.
tmpfs Mounts	Security / Speed	Stored only in host memory; never written to disk.	Sensitive keys, temporary fast cache data.

Volume Commands

`docker volume create my-vol`
`docker volume ls`
`docker volume rm my-vol`

```
# Modern mount syntax (preferred over -v for clarity):
docker run -d   --name db   --mount type=volume,source=my-data,target=/var/lib/
postgresql/data   postgres:15
```

09 DOCKERFILE CHEAT SHEET

Instruction	Purpose	Example & Tip
`FROM`	Sets the base image. Must be the first instruction.	`FROM node:18-alpine` *Tip: Use minimal images like alpine.*
`WORKDIR`	Sets the working directory for subsequent instructions.	`WORKDIR /app` *Tip: Prefer this over `RUN cd /app`.*
`COPY`	Copies files from host to image.	`COPY package.json .` *Tip: Copy dependency files first for caching.*
`ADD`	Like COPY, but extracts tarballs and fetches URLs.	`ADD https://example.com/file.tar.gz /` *Tip: Always prefer COPY unless extracting.*
`RUN`	Executes commands in a new layer during the build.	`RUN npm ci --only=production` *Tip: Chain commands with `&&` to reduce layers.*
`ENV`	Sets persistent environment variables.	`ENV NODE_ENV=production`
`ARG`	Build-time variables (not available at runtime).	`ARG VERSION=1.0`
`EXPOSE`	Documents intended ports (does NOT actually publish).	`EXPOSE 8080`
`USER`	Sets the UID/username for subsequent commands.	`USER node` *Tip: Never run apps as root in production.*
`CMD`	Default command to run when container starts.	`CMD ["node", "server.js"]` *Tip: Can be overridden at runtime.*
`ENTRYPOINT`	Configures container to run as an executable.	`ENTRYPOINT ["nginx", "-g", "daemon off;"]`

CMD vs ENTRYPOINT

ENTRYPOINT sets the main executable (cannot be easily overridden).
CMD provides default arguments to the ENTRYPOINT (can be easily overridden).
Combined: `ENTRYPOINT ["python", "app.py"]` + `CMD ["--help"]`

10 BUILD & MULTI-STAGE WORKFLOWS

Modern Build Commands (BuildKit)

- `docker build -t app .` : Standard build.
- `docker buildx build --platform linux/amd64,linux/arm64 -t app:v1 --push .` : Multi-platform build.
- `docker build --no-cache .` : Force rebuild ignoring cache.

The .dockerignore File

Prevents unnecessary or sensitive files from entering the build context, speeding up builds and reducing image size.

```
node_modules
.git
.env
*.log
Dockerfile*
README.md
```

BUILD FASTER: Optimization Tips

- **Order matters:** Place least-frequently changed instructions at the top (e.g., install dependencies before copying source code) to maximize layer caching.
- **Minimize layers:** Chain `apt-get update && apt-get install`.
- **Clean up:** Remove apt caches (`rm -rf /var/lib/apt/lists/*`) in the same RUN step.

Multi-Stage Builds

Use multi-stage builds to compile code in a heavy environment, but copy only the compiled binary to a tiny, secure production image.

```
# Stage 1: Build environment
FROM golang:1.20 AS builder
WORKDIR /app
COPY go.mod go.sum ./
RUN go mod download
COPY . .
RUN CGO_ENABLED=0 GOOS=linux go build -o main .

# Stage 2: Production environment (tiny image)
FROM alpine:latest
WORKDIR /root/
COPY --from=builder /app/main .
EXPOSE 8080
CMD ["./main"]
```

11 DOCKER COMPOSE CHEAT SHEET

Docker Compose is a tool for defining and running multi-container applications using a YAML file. *Note: Use the modern* `docker compose` *syntax (V2), not the legacy* `docker-compose`.

Command	Action
`docker compose up -d`	Build, create, start, and detach.
`docker compose down`	Stop and remove containers, networks.
`docker compose logs -f`	Follow log output for all services.
`docker compose ps`	List containers for the project.
`docker compose build`	Force rebuild of service images.
`docker compose exec <svc> sh`	Execute command in a service.

compose.yaml Example

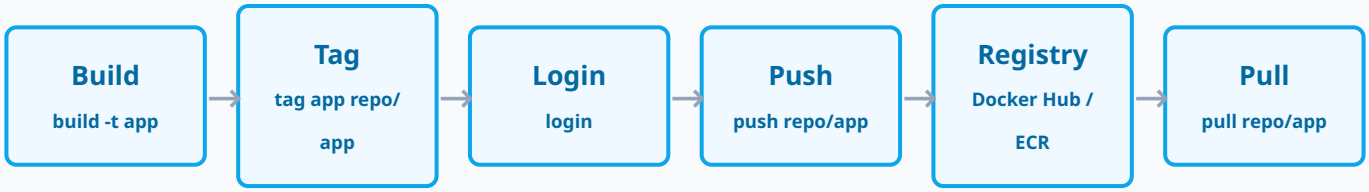
```
services:
  backend:
    build: ./backend
    ports:
      - "8080:8080"
    environment:
      - DB_HOST=db
      - DB_USER=${DB_USER} # Loaded from .env
    depends_on:
      db:
        condition: service_healthy

  db:
    image: postgres:15-alpine
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U
postgres"]
      interval: 10s
      retries: 5

volumes:
  pgdata:
```

12 ENVIRONMENT VARIABLES & SECRETS

Type	Availability	Usage
ARG	Build-time only	Versions, build flags.
ENV	Build & Runtime	Paths, simple configs.
Secrets	Runtime (Memory)	Passwords, API keys.

CRITICAL SECURITY WARNING: Never bake passwords, API keys, private keys, or credentials into Docker images using `ENV` or hardcoded files. Anyone who pulls the image can extract them. Use Docker Secrets or inject them at runtime via orchestrators.

13 REGISTRY OPERATIONS



14 CLEANUP, RESOURCES & HEALTH

Clean Your Environment (Pruning)

Warning: Prune commands are destructive. Ensure you understand what is being removed.

- `docker container prune` : Remove stopped containers.
- `docker image prune` : Remove dangling images.
- `docker volume prune` : Remove unused volumes.
- `docker system prune -a --volumes` : **Nuke it all.** Removes everything not currently used by a running container.

Resource Limits

Without limits, a container can consume all host CPU and Memory, causing host instability.

`docker run --memory="512m" --cpus="1.0" myapp`

- `--memory="1g"` : Hard limit on RAM.
- `--memory-reservation="512m"` : Soft limit.
- `--cpus="1.5"` : Limit to 1.5 CPU cores.

Healthchecks

A container running does not mean the app is healthy (e.g., deadlocked web server). Healthchecks tell Docker how to test the app.

```
HEALTHCHECK --interval=30s --timeout=5s --start-period=10s --retries=3   CMD curl -f
http://localhost:8080/health || exit 1
```

States: `starting` → `healthy` (if CMD returns 0) → `unhealthy` (if CMD returns 1).

15 SECURITY & PRODUCTION PRACTICES

SECURITY DO's

- ☑ Use trusted, minimal base images (Alpine, Distroless).
- ☑ Run containers as a non-root user (`USER node`).
- ☑ Use Multi-stage builds to reduce attack surface.
- ☑ Apply resource limits (Memory & CPU).
- ☑ Use read-only filesystems (`--read-only`) when possible.
- ☑ Scan images for vulnerabilities (Docker Scout, Trivy).

SECURITY DON'Ts

- ✗ Run everything as root.
- ✗ Store secrets in Dockerfiles or images.
- ✗ Expose databases publicly via port mapping without need.
- ✗ Treat containers as robust security boundaries by default (they share the kernel).
- ✗ Install unnecessary tools (curl, ssh) in production images.

Production Readiness Checklist

- ☑ Small, optimized images
- ☑ Multi-stage builds utilized
- ☑ Non-root user configured
- ☑ Health checks defined
- ☑ CPU & Memory limits set
- ☑ Secrets injected securely at runtime
- ☑ Image passed vulnerability scan
- ☑ Predictable versioning (No `latest`)
- ☑ Minimal exposed ports
- ☑ Logging driver configured
- ☑ Graceful shutdown handled (SIGTERM)
- ☑ Restart strategy defined

BUILD → TEST → SCAN → TAG → PUSH → DEPLOY → MONITOR

16 TOP 10 DOCKER MISTAKES

1. Using `latest` everywhere

Breaks determinism. Update overwrites image silently. *Fix: Pin versions.*

2. Running as Root

Huge security risk if container escapes. *Fix: Use* `USER` *instruction.*

3. Bloated Images

Slow pulls, large attack surface. *Fix: Alpine, Multi-stage builds.*

4. Ignoring `.dockerignore`

Copies node_modules, secrets, git cache. *Fix: Always add a .dockerignore.*

5. Hardcoding IP Addresses

Container IPs change. *Fix: Use Docker Network & service names.*

6. Baking Secrets

ENV variables in Dockerfile leak secrets. *Fix: Mount secrets at runtime.*

7. Treating Containers as Storage

Data dies with the container. *Fix: Use Docker Volumes.*

8. No Healthchecks

Orchestrators assume deadlocked apps are fine. *Fix: Add HEALTHCHECK.*

9. Zombie Processes

PID 1 doesn't forward signals. *Fix: Use* `tini` *or exact exec array syntax.*

10. Monolithic Containers

Running DB + App + Nginx in one container. *Fix: One concern per container.*

17 TROUBLESHOOTING MATRIX

Problem	First Command	What to Check
Container exits immediately	`docker logs <id>`	Application syntax error, missing environment variables, crashing process.
Port unavailable / App unreachable	`docker ps`	Is the container port correctly mapped to the host? (`-p 8080:80`). Is firewall blocking?
Services cannot communicate	`docker network inspect`	Are they on the same custom bridge network? Are they using service names, not IPs?
Data disappeared after restart	`docker inspect <id>`	Was a volume or bind mount actually attached? (Check Mounts section).
Image size is massive	`docker history <id>`	Identify the layer causing bloat. Are dependencies being compiled without multi-stage?
Build is extremely slow	Build output	Is `.dockerignore` missing? Are you copying source code before `npm install`?
Container is "unhealthy"	`docker inspect <id>`	Check the Healthcheck State log. Is the endpoint `/health` returning 200?
Permission Denied inside container	`docker exec -it <id> sh`	Is the USER lacking filesystem permissions for the mounted Volume?

18 COMMANDS ULTRA-QUICK REFERENCE

SYSTEM & INFO

```
docker info
docker version
docker system df
docker login
```

IMAGES

```
docker pull <image>
docker build -t <name> .
docker images
docker rmi <image>
```

DEBUGGING

```
docker logs -f <id>
docker exec -it <id> sh
docker top <id>
docker stats
```

CONTAINERS

```
docker run -d -p 80:80 <img>
docker ps -a
docker stop <id>
docker start <id>
docker rm -f <id>
```

NETWORK

```
docker network ls
docker network create <n>
docker network inspect <n>
docker network connect <n>
<id>
```

COMPOSE

```
docker compose up -d
docker compose down
docker compose logs -f
docker compose ps
```

VOLUMES & CLEANUP

```
docker volume create <v>
docker volume ls
docker system prune -a
docker volume prune
```

19 INTERVIEW BOOSTER Q&A

Q: Difference between Image and Container?
A: Image is an immutable blueprint (code, bins, libs). Container is the running, mutable instance of that image.

Q: CMD vs ENTRYPOINT?
A: ENTRYPOINT sets the fixed executable. CMD provides default arguments. ENTRYPOINT is hard to override; CMD is overridden easily by passing args to `docker run`.

Q: COPY vs ADD?
A: Both copy files into the image. ADD has extra features (extracts tarballs, fetches URLs). Best practice is to use COPY unless extracting.

Q: Volume vs Bind Mount?
A: Volumes are fully managed by Docker and isolated from core host OS files. Bind mounts rely on the exact host directory structure and path.

Q: Why use a multi-stage build?
A: It allows you to use a heavy base image to compile code, then copy only the compiled artifact to a minimal base image, drastically reducing final image size and attack surface.

INTERVIEW TIP: Know the difference between `EXPOSE` (documentation only) and `-p` (actual port mapping/publishing to host).

KEEP LEARNING.
KEEP BUILDING.

Master the tools. Build real projects. Become industry-ready.

AlgoTutor

Learn • Build • Practice • Grow

AlgoTutor provides practical, high-quality learning resources for developers, DevOps engineers, and aspiring technology professionals.

☑ Engineering Roadmaps

☑ Technical Cheat Sheets

☑ Interview Preparation Guides

☑ Practical Career Resources

FOLLOW ALGOTUTOR ON LINKEDIN